GeoCrop: Base Architecture vs. Old Prototype

Which technology replaces which, and how the change affects performance

F26-155-GeoCrop — Measured on AOI1 (eastern Washington, crop-type layer) — October 2026

Layer	Old prototype	Base architecture (replacement)	Effect on performance
Web framework / backend	Django: renders pages on the server and also serves data	**FastAPI** (REST API only)	Async API returns small JSON (catalogue, stats); pages no longer rebuilt per request
Frontend	Django templates + plain JavaScript	**React** (Vite build) served by **Nginx**	Built once into static files; Nginx serves them with compression and caching
Map library	Leaflet 1.9 drawing every shape (SVG / canvas)	**MapLibre GL JS** (WebGL, via react-map-gl)	GPU draws tiles; many layers stay smooth while zooming and panning
Model results on the map	One 43.3 MB GeoJSON file with 101,435 polygons (∼750k vertices), sent whole	**Rasters as Cloud-Optimised GeoTIFF** served by **TiTiler** as map tiles	Only visible tiles are sent: **337 KB per screen** instead of 43.3 MB (∼130× less)
Field boundaries	Same large GeoJSON	**PMTiles** vector tiles (1.86 MB file, read by byte range)	Browser fetches only the tiles in view; no full-file download or parse
Database	SQLite file; sample statistics hand-written in Python	**PostgreSQL + PostGIS**	Spatial queries and real statistics computed from results; safe for many users
Processing of uploads	Upload converted to one PNG preview; no model run	**Celery** workers with **Redis** queue; model registry	Models run in the background, block by block; the map and API stay responsive; **57 s** upload-to-map
File storage	Files on the local disk of the web server	**MinIO** (S3-compatible object storage), signed upload URLs	Uploads go straight to storage, not through the API; tiles read files by byte range
Caching	None	**Nginx tile cache**	Repeat tiles served in **1 ms**
Models	Hard-wired scripts	**Plug-in model registry** (folder + manifest)	New models added without changing API, database or interface
Deployment	Manual Python setup on one machine	**Docker Compose**: 7 services, one command	Same stack on every machine; services scale or restart independently

Measured effect (AOI1, crop-type layer, zoom 12)

Measure	Old prototype	Base architecture
Data for one screen	43.3 MB	**337 KB** (24 tiles)
Objects the browser draws	101,435 polygons	24 tile images
Server / parse time	1,021 ms just to parse JSON	cold tile 47 ms median (p95 54 ms)
Repeat view	Re-download or re-parse	cached tile **1 ms**
Stored result size	43.3 MB GeoJSON	222 KB compressed GeoTIFF
Uploaded image	PNG preview only	analysed and on map in **57 s**

Why it is faster, in short

- **Send only what is visible.** Tiles replace whole files, so data per screen stays small as images and models grow.
- **Keep results as rasters.** A pixel map is far smaller than polygons and needs no conversion.
- **Do heavy work in the background.** Queued workers run models; the website never waits.
- **Cache repeated work.** Nginx keeps tiles that were already drawn.
- **Draw with the GPU.** MapLibre renders tiles with WebGL instead of thousands of shapes.